

BANK MANAGEMENT SYSTEM

A Detailed Walkthrough of the Oracle SQL Project



Schema · Constraints · DML · Functions · Joins · Window Functions · CTEs

Modeling a Bank, End to End

This project builds a small banking database from scratch and uses it to practice the full range of practical SQL skills — from schema design and data integrity through to advanced querying with window functions and CTEs. Every query runs against real, self-inserted sample data rather than toy examples, and the design decisions (which columns are required, which values are restricted, how tables relate) mirror the kind of thinking a real banking system would need.

- Design a normalized schema with correct relationships
- Enforce data integrity through constraints
- Practice transaction control safely (COMMIT / ROLLBACK / SAVEPOINT)
- Answer real business questions using SQL

PROJECT SCALE

5

TABLES

8

PROJECT PARTS

25+

TRANSACTIONS

10+

CUSTOMERS

Five Tables, One Bank

1

CUSTOMER

customer_id, customer_name, phone, email, city

2

BRANCH

branch_id, branch_name, city

3

ACCOUNT

account_id, customer_id, branch_id, account_type, balance, opening_date

4

EMPLOYEE

employee_id, employee_name, branch_id, designation, salary

5

BANK_TRANSACTION

transaction_id, account_id, transaction_type, amount, transaction_date

CUSTOMER

PURPOSE

Stores the identity and contact details of every person who banks with this institution. This is one of the two tables (along with BRANCH) that everything else in the schema ultimately connects back to.

RELATIONSHIP

One customer can hold many accounts, but every account belongs to exactly one customer — a one-to-many relationship, enforced by the customer_id foreign key on ACCOUNT.

Column	Constraint
customer_id	PRIMARY KEY
customer_name	NOT NULL
phone	NOT NULL, UNIQUE
email	UNIQUE
city	DEFAULT 'Unknown'

BRANCH

PURPOSE

Represents each physical branch location the bank operates. Small in structure, but it anchors two other tables — every account is opened at a branch, and every employee is staffed at one.

RELATIONSHIP

One branch can have many accounts and many employees, but each account and each employee points back to exactly one branch_id — again, one-to-many in both directions.

Column	Constraint
branch_id	PRIMARY KEY
branch_name	NOT NULL, UNIQUE
city	NOT NULL

ACCOUNT

PURPOSE

The central table of the schema. It doesn't just record a balance — it's the join point between a customer and a branch, and it's what every transaction is ultimately tied to.

RELATIONSHIP

ACCOUNT holds two foreign keys at once: customer_id (which customer owns it) and branch_id (where it was opened). BANK_TRANSACTION then references account_id, making ACCOUNT the hub of the whole schema.

Column	Constraint
account_id	PRIMARY KEY
customer_id	FOREIGN KEY → CUSTOMER
branch_id	FOREIGN KEY → BRANCH
account_type	CHECK (Savings/Current/FD)
balance	CHECK (>= 0), DEFAULT 0
opening_date	DEFAULT SYSDATE

EMPLOYEE

PURPOSE

Records the bank's staff — who works where, in what role, and at what salary. Kept separate from CUSTOMER since employees and customers are fundamentally different entities with different attributes.

RELATIONSHIP

Many employees can work at the same branch, but each employee record points to exactly one branch_id — a straightforward one-to-many relationship from BRANCH to EMPLOYEE.

Column	Constraint
employee_id	PRIMARY KEY
employee_name	NOT NULL
branch_id	FOREIGN KEY → BRANCH
designation	NOT NULL, DEFAULT 'Clerk'
salary	CHECK (salary > 0)

BANK_TRANSACTION

PURPOSE

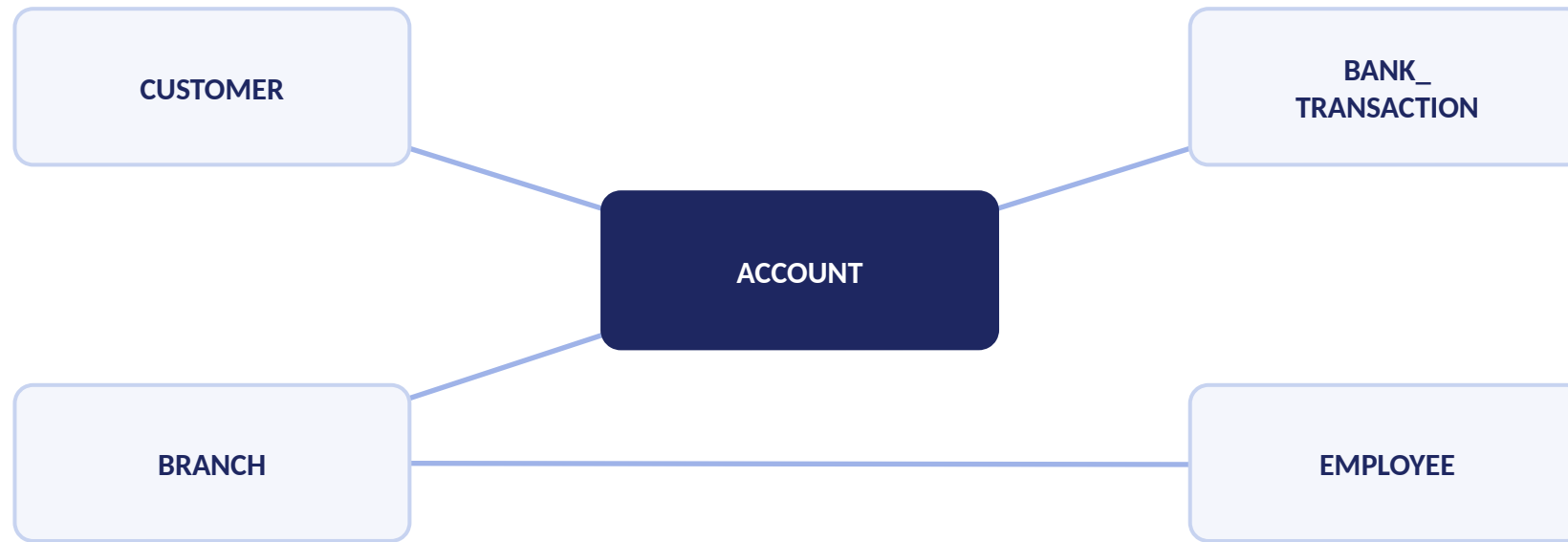
Logs every deposit, withdrawal, and transfer that happens on an account. This table grows the fastest in a real bank, and it's the one most of the reporting and analysis in later parts of the project is built on.

RELATIONSHIP

Every transaction belongs to exactly one account, so `account_id` is a foreign key back to `ACCOUNT`. An account, in turn, can have many transactions over its lifetime.

Column	Constraint
transaction_id	PRIMARY KEY
account_id	FOREIGN KEY → ACCOUNT
transaction_type	CHECK (Deposit/Withdrawal/Transfer)
amount	CHECK (amount > 0)
transaction_date	DEFAULT SYSDATE

How the Tables Connect



PRIMARY KEY

Uniquely identifies each row in its own table. Cannot be NULL or duplicated.

FOREIGN KEY

A column that references another table's primary key, enforcing that the value must already exist there.

Constraints applied throughout:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

DEFAULT

Constraint Types, Explained

1 NOT NULL

Every row must have a value in this column — it can never be left blank.

2 UNIQUE

No two rows may share the same value in this column (duplicates rejected).

3 CHECK

The value must satisfy a stated condition — e.g. `balance >= 0`, or `salary > 0`.

4 DEFAULT

If no value is supplied on INSERT, Oracle fills in this value automatically.

5 PRIMARY KEY

Combines NOT NULL and UNIQUE — the single column (or set) that identifies a row.

6 FOREIGN KEY

Ties a column's values back to another table's primary key, preventing orphaned records.

Eight Parts, One SQL Skill Set

1

Table Creation

CREATE TABLE, constraints, ALTER TABLE

2

DML & Transactions

INSERT, UPDATE, DELETE, COMMIT,
ROLLBACK, SAVEPOINT

3

SELECT & Filtering

WHERE, operators, IN, BETWEEN, LIKE,
ORDER BY

4

SQL Functions

String, numeric, date functions, CASE, NVL

5

Aggregates & Grouping

COUNT, SUM, AVG, GROUP BY, HAVING

6

Joins

Inner & outer joins across all five tables

7

Window Functions

RANK, ROW_NUMBER, PARTITION BY,
LAG/LEAD

8

CTEs

WITH clauses for layered, readable reports

DML & Transaction Control

THE THREE DML STATEMENTS

INSERT

Adds a new row to a table.

UPDATE

Changes existing values in one or more rows matching a condition.

DELETE

Removes rows matching a condition (the table itself stays).

TRANSACTION CONTROL

SAVEPOINT

Marks a point in a transaction you can roll back to, without undoing everything before it.

ROLLBACK

Undoes changes — either back to a named SAVEPOINT, or entirely back to the last COMMIT.

COMMIT

Makes all changes since the last COMMIT permanent and visible to other sessions.

WHY THIS MATTERS FOR THE PROJECT

Oracle does not auto-commit by default in most tools, so changes only become permanent once COMMIT runs. This lets you safely test an UPDATE or DELETE, use ROLLBACK to undo it if something looks wrong, and only COMMIT once the data is verified — leaving the final database in the consistent state the manual asks for.

Querying, Filtering & Functions

FILTERING WITH WHERE

- Comparison: =, <>, >, <, >=, <=
- Logic: AND (all true), OR (any true), NOT (negate)
- IN — match against a list of values
- BETWEEN — match an inclusive range
- LIKE — partial match using % and _ wildcards
- ORDER BY — sort results ASC or DESC

BUILT-IN FUNCTIONS

UPPER / LOWER	Change text case
LENGTH / SUBSTR	String length / extract part of a string
ROUND / MOD	Round a number / remainder of division
TO_CHAR	Format a date or number as text
NVL	Substitute a value when a column is NULL
CASE	Return different labels based on conditions

Oracle evaluates a query in this order: FROM → WHERE → SELECT → ORDER BY — the opposite of how it's written. That's why a column alias defined in SELECT can't be referenced inside WHERE.

Aggregates vs. Window Functions

AGGREGATE FUNCTIONS

COUNT • SUM • AVG • MAX • MIN

Collapse many rows into one summary row per group. GROUP BY defines the groups; HAVING filters groups after aggregation (WHERE can't do this, since it runs before grouping).

Example question: "What's the total balance for each account type?" — one row comes back per account type.

WINDOW FUNCTIONS

RANK • ROW_NUMBER • LAG/LEAD • running totals

Calculate across a set of rows using OVER(), but keep every individual row visible — nothing collapses. PARTITION BY resets the calculation per group, similar in spirit to GROUP BY but without merging rows.

Example question: "What's each account's rank by balance?" — every account still shows, now with a rank column.

Joins and CTEs

JOIN TYPES

INNER JOIN

Returns only rows with a match in both tables.

LEFT JOIN

Keeps every row from the left table, filling NULLs where there's no match on the right.

RIGHT JOIN

The mirror of LEFT JOIN — keeps every row from the right table instead.

WHAT A CTE IS

A CTE (WITH clause) names a subquery so it can be reused and read top-to-bottom, like a temporary, named result set that only exists for one statement.

WHY USE ONE INSTEAD OF A NESTED SUBQUERY?

Readability. A CTE breaks a complex query into named, logical steps instead of nesting parentheses inside parentheses — easier to write, debug, and explain.

In this project, joins connect CUSTOMER → ACCOUNT → BRANCH → EMPLOYEE → BANK_TRANSACTION for combined reports, and CTEs pre-summarize data (like totals per account) before joining it back for a final report.

From Schema to Insight

This project ties together design, integrity, and analysis — a single database taken from raw CREATE TABLE statements to ranked, aggregated, multi-table reports.

[github.com](#) — full schema, sample data, and queries